Efficient MPI Circle Rendering with Adaptive Heuristic Decomposition Methods

An-Che Liang, Guan-Ming Chiu, Pei-Yu Wu
National Taiwan University
Team 12

1 Environment

The experiments were conducted on Ubuntu 24.04.1 LTS running Linux kernel 6.8.0-100-generic. The host system used an Intel Xeon Platinum 8352V processor at 2.10 GHz with a topology of 2 sockets, 36 cores per socket, 2 threads per core, and 2 NUMA nodes. The MPI runtime was Open MPI 4.1.6, and the compiler was `mpicc` backed by GCC 13.3.0.

Table 1: Experimental configuration used for benchmarking.

Setting	Value
Build command	`mpicc renderer_mpi.c -o renderer_mpi -lm -march=native`
Process counts	1, 4, 8, 16, 64
Single-node limit	Up to 16 MPI processes
64-process configuration	4 nodes with 16 processes per node via `hosts`
Reported time	Median of 10 runs
Timing definition	Total wall time printed by `renderer_mpi`

2 Implementation

The final MPI renderer selects between two strategies at runtime according to the number of circles. Scenes with 500,000 circles or fewer use Mode A, named REPLICATE, whereas scenes above that threshold use Mode B, named PARTITION. The threshold was tuned empirically against benchmark data at 64 processes and can be overridden through the `RENDERER_THRESHOLD` environment variable.

2.1 Mode A: REPLICATE

In Mode A, rank 0 reads the CRDR file and broadcasts all circle records to every rank. Each rank owns a cyclic stripe of image rows, where rank r owns rows $r, r + P, r + 2P, \ldots$, with P denoting the process count. Before rasterization, each rank prefilters the full record list to retain only circles whose bounding-box vertical range intersects its owned rows. The prefiltered list caches the bounding box and the first owned row, denoted y_{first}, so the inner raster loop remains branch-free with respect to ownership. Each rank rasterizes into a compact $W \times \text{local\_rows}$ RGB buffer, and `MPI_Gatherv` collects the compact slices before rank 0 reinterleaves them into the final image.

Cyclic row ownership balances work even when circles cluster in a vertical region, which a contiguous block assignment cannot guarantee. The compact per-rank buffer also avoids allocating a full $W \times H$ image on every process, which becomes increasingly important as P grows.

Table 2: Summary of benchmark test cases.

Testcase	Circles	Image	Mean radius	Max radius
`imbalance_c100000`	100,000	320×240	25.62	1998.10
`medium_c200000`	200,000	640×480	10.49	20.00
`large_c1000000`	1,000,000	640×480	10.50	20.00
`large_c2000000`	2,000,000	640×480	10.50	20.00
`large_c4000000`	4,000,000	640×480	10.50	20.00

2.2 Mode B: PARTITION

In Mode B, rank 0 reads and broadcasts the full record array. A scatter-based distribution was measured to be approximately three times slower than broadcast on this cluster for equivalent payloads, so broadcast is used in both modes. Each rank then slices the record array locally: rank r owns a contiguous chunk of $count/P$ consecutive records in file order, with rank 0 holding the earliest, back-most circles and rank $P-1$ holding the latest, front-most circles. Each rank rasterizes its own records into a full $W \times H$ buffer of `uint32_t`-encoded pixels, where each pixel is packed as $((\text{rank}+1) \ll 24) \mid R \ll 16 \mid G \ll 8 \mid B$. Untouched pixels remain zero. A collective `MPI_Reduce` with `MPI_MAX` across all ranks then selects the correct color per pixel, because a larger rank marker indicates that a later circle should win under the back-to-front painter semantics. Using the predefined `MPI_MAX` operation on `MPI_UINT32_T` instead of a custom non-commutative `MPI_Op` keeps the reduction on the hardware-accelerated collective path.

This mode reduces the per-rank rasterization cost to approximately $1/P$ of the serial work, at the cost of a full-image $W \times H \times 4$-byte buffer on each rank and a reduction across all pixels.

2.3 Common optimizations

The implementation applies per-row analytic x-range trimming with `sqrtf`, followed by the exact circle-inside test, to avoid iterating over pixels that are clearly outside the circle. The renderer writes opaque RGB values directly, which matches the assignment format in which records contain only `uint8_t` red, green, and blue components. The source also uses `#pragma GCC optimize("O3", "unroll-loops")` because the official compile command does not specify an explicit optimization flag.

3 Test Cases

The binary metadata in `data/testcase_metadata.csv` shows that the large cases use a 640×480 image with 1M, 2M, and 4M circles. The special `imbalance_c100000` case is smaller, using a 320×240 image with 100K circles, but it has much larger radius variation. Its maximum radius is approximately 1998 pixels, compared with roughly 20 pixels for the regular large cases. This makes the case suitable for stressing load balance.

4 Benchmark Results

The benchmark uses the median wall time across 10 repetitions for each testcase and process-count pair. The one-process MPI run serves as the serial baseline for speedup and efficiency because it exercises the same optimized renderer with $P = 1$.

The best 64-process strong-scaling result is `large_c2000000`, which improves from 3.336 s on one process to 0.252 s on 64 processes, corresponding to a 13.24$\times$ speedup. On a single local node, the same case improves to 0.284 s on 16 processes, which is an 11.75$\times$ speedup with 73.4% efficiency. The smaller cases still improve, but their efficiency declines because fixed costs such as record broadcast, gather, PNG writing, MPI launch overhead, and multi-node communication occupy a larger share of total runtime.

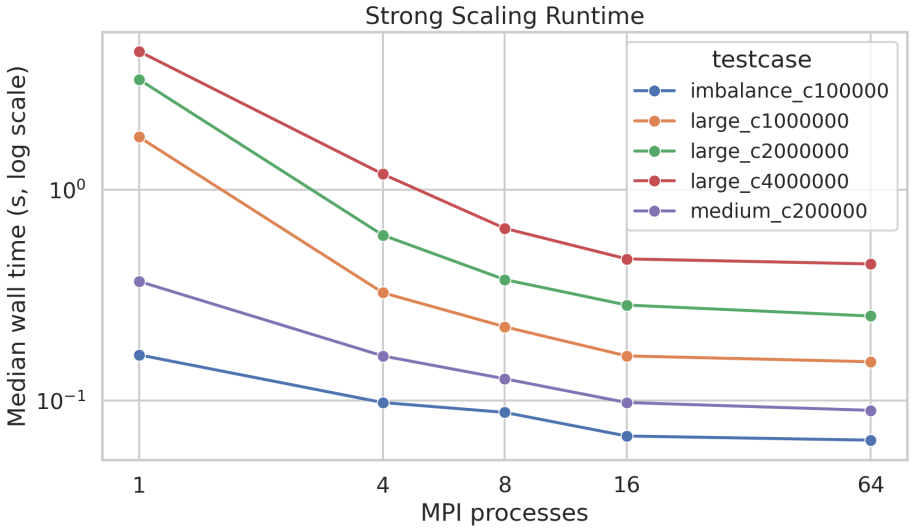

Figure 1: Strong-scaling runtime across process counts.

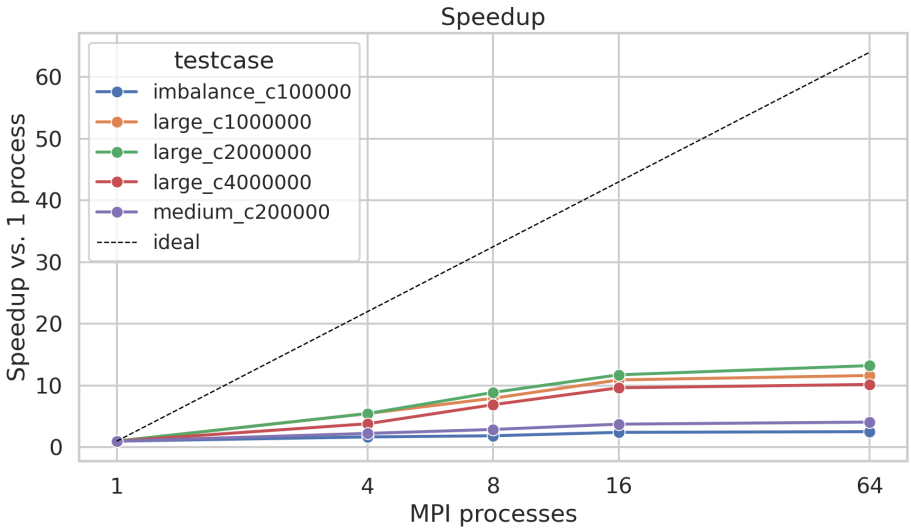

Figure 2: Speedup across process counts.

5 Problem Size Per Process

At 64 processes, throughput increases with larger scenes: 1.54 million circles per second for `imbalance_c100000`, 2.22 million circles per second for `medium_c200000`, 6.54 million circles per second for `large_c1000000`, 7.94 million circles per second for `large_c2000000`, and 8.99 million circles per second for `large_c4000000`. These values are reflected in `data/summary_64proc.csv` and `figure/problem_size_per_process.png`.

The weak-scaling conclusion is mixed. More circles per process amortize fixed MPI and output costs, so throughput improves as the problem grows. However, wall time still increases from 0.153 s at 1M circles to 0.445 s at 4M circles on 64 processes because each process still loops over every circle and skips only the pixels outside its owned rows. The algorithm scales well for strong scaling, but it is not true weak scaling because both broadcast size and per-rank circle iteration grow with total circle count.

Table 3: Median runtime, speedup, and efficiency for each testcase.

Testcase	1 proc (s)	4 proc (s)	8 proc (s)	16 proc (s)	64 proc (s)	64-proc speedup	64-proc efficiency
`imbalance_c100000`	0.165	0.098	0.088	0.068	0.065	2.54×	4.0%
`medium_c200000`	0.368	0.163	0.127	0.098	0.090	4.09×	6.4%
`large_c1000000`	1.780	0.325	0.224	0.163	0.153	11.63×	18.2%
`large_c2000000`	3.336	0.609	0.375	0.284	0.252	13.24×	20.7%
`large_c4000000`	4.534	1.186	0.656	0.470	0.445	10.19×	15.9%

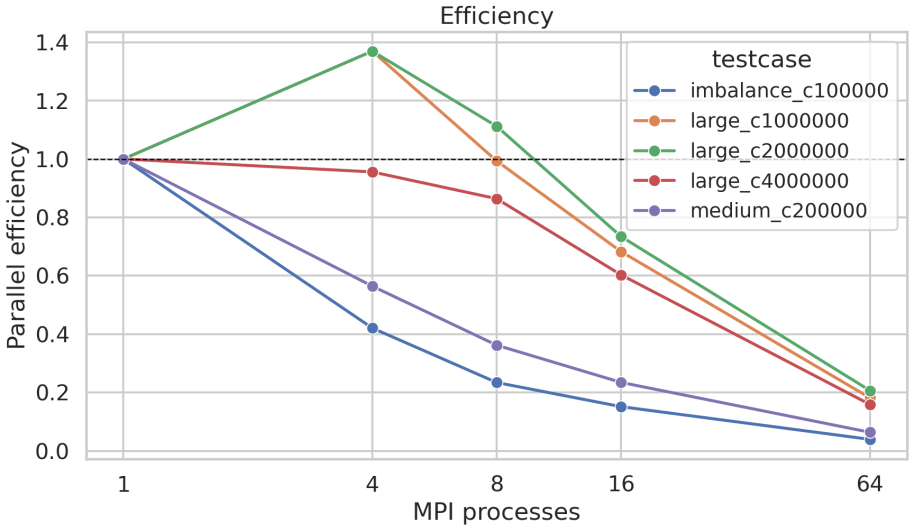

Figure 3: Parallel efficiency across process counts.

6 Load Balance

The renderer logs per-rank render time statistics and reports imbalance as $\max/\text{mean}$. The measured imbalance values are recorded in `data/benchmark_results.csv` and plotted in `figure/imbalance.png`.

The three large scenes, namely `large_c1000000`, `large_c2000000`, and `large_c4000000`, use Mode B. In this mode each rank renders a disjoint slice of records into a full image, so imbalance reflects variation in circle counts across ranks. Because records are split evenly, imbalance arises from radius variation rather than spatial clustering. At 16 processes the imbalance is 1.21, 1.23, and 1.10 respectively, and at 64 processes it is 1.28, 1.48, and 1.27. These values remain controlled because the large scenes use uniformly sized circles with a maximum radius of 20 pixels.

The two smaller scenes, `imbalance_c100000` and `medium_c200000`, use Mode A. In this mode each rank rasterizes prefiltered circles into its cyclic row stripe, so imbalance reflects whether very large circles disproportionately cover certain rows. At 64 processes, `medium_c200000` has imbalance 1.38, and `imbalance_c100000` has the highest imbalance at 1.64. The special imbalance case contains circles with radii up to 1998 pixels in a 320×240 image, so a single circle can span almost the full image height. Cyclic row distribution spreads that coverage across all ranks rather than concentrating it in one contiguous block, which keeps imbalance below $2\times$ despite the extreme radius variation. Even with this imbalance, wall time improves from 0.165 s on one process to 0.065 s on 64 processes.

7 Conclusion

The MPI renderer is correct on all benchmarked test cases and demonstrates meaningful strong scaling under the 64-process competition configuration. The adaptive strategy, which switches from cyclic row decomposition in Mode A for smaller scene sizes to record partitioning with a painter

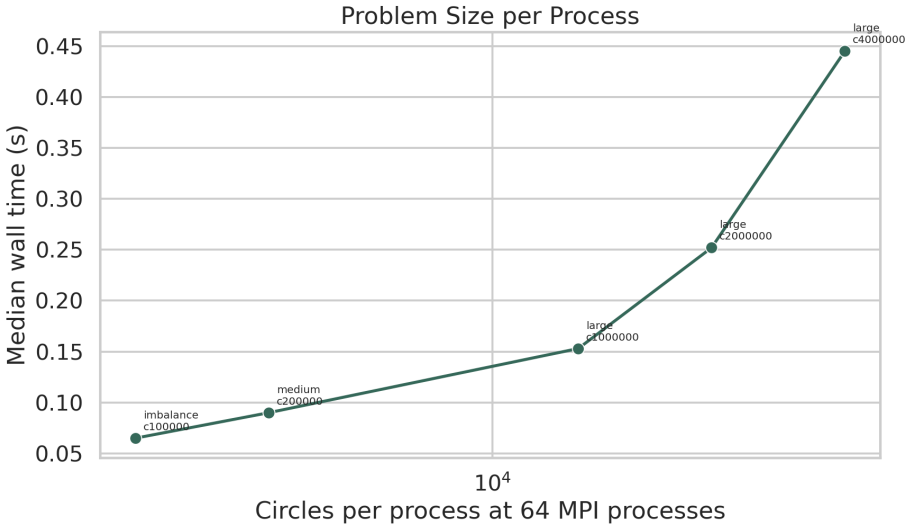

Figure 4: Throughput as a function of problem size at 64 processes.

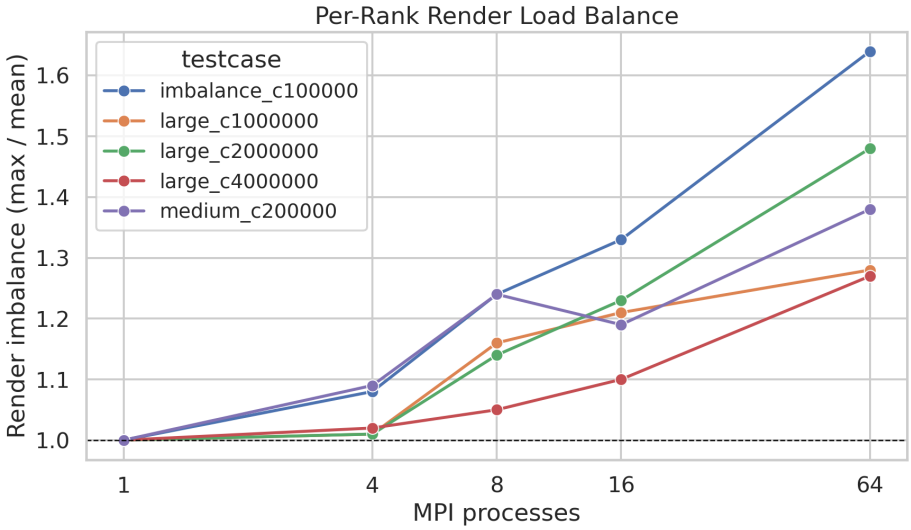

Figure 5: Per-rank render imbalance across benchmark cases.

reduction in Mode B for larger scene sizes, provides good performance across both regimes. The best measured result is a 13.24× speedup on `large_c2000000` under Mode B, while the 4M-circle case achieves the highest throughput at 8.99 million circles per second.

The main cost in Mode A is that every rank scans all records during prefiltering, so broadcast size and prefilter time grow with total circle count regardless of P. Mode B addresses this by giving each rank only $count/P$ records to rasterize, but it trades that improvement for a full-image `uint32_t` buffer per rank and a pixel-level `MPI_MAX` reduction. The threshold of 500,000 circles was chosen where the reduction overhead in Mode B becomes cheaper than the per-rank prefilter scan cost in Mode A at 64 processes.